

Complete Machine Learning Training Guide

End-to-End Model Training Across All Domains

Table of Contents

1. [Universal ML Pipeline](#)
 2. [Traditional Machine Learning](#)
 3. [Deep Learning](#)
 4. [Natural Language Processing](#)
 5. [Large Language Models](#)
 6. [Computer Vision](#)
 7. [Common Parameters Explained](#)
 8. [Debugging & Troubleshooting](#)
-

Universal ML Pipeline

Every machine learning project follows these core steps, regardless of domain:

1. Problem Definition & Data Understanding

Why this matters: Clear problem definition drives all subsequent decisions

- **Define success metrics:** What constitutes a good model for your use case?
- **Understand data distribution:** Plot histograms, check correlations
- **Identify data quality issues:** Missing values, outliers, inconsistencies
- **Check class balance:** Imbalanced datasets need special handling

Key Questions to Ask:

- Is this supervised or unsupervised learning?
- Classification or regression?
- How much data do I have?
- What's the business impact of false positives vs false negatives?

2. Data Preprocessing Pipeline

Why preprocessing is crucial: Models perform poorly on raw, messy data

Data Cleaning

- **Missing values:**

- Drop rows/columns if <5% missing
- Impute with mean/median for numerical
- Mode or "Unknown" category for categorical
- **Outliers:** Use IQR method or domain knowledge to handle
- **Duplicates:** Remove exact duplicates, be careful with near-duplicates

Feature Engineering

- **Numerical features:**
 - Create polynomial features for non-linear relationships
 - Bin continuous variables if needed
 - Log transform for right-skewed distributions
- **Categorical features:**
 - One-hot encoding for nominal (no order)
 - Label encoding for ordinal (has order)
 - Target encoding for high-cardinality categories
- **Temporal features:**
 - Extract day, month, year, weekday from dates
 - Create time-since features
 - Seasonal decomposition

Feature Scaling

Why scale: Algorithms like SVM, KNN sensitive to feature magnitudes

- **StandardScaler:** Mean=0, Std=1 (use when features normally distributed)
- **MinMaxScaler:** Scale to [0,1] (use when you want bounded range)
- **RobustScaler:** Less sensitive to outliers (use with outliers present)

3. Data Splitting Strategy

Why proper splitting matters: Prevents data leakage and overoptimistic results

Time Series Data

- **Never shuffle:** Use temporal splits
- **Walk-forward validation:** Train on past, validate on future
- **Gap between train/test:** Account for data processing delays

Regular Data

- **Stratified split:** Maintains class distribution across splits
 - **Common ratios:** 70/15/15 or 80/10/10 (train/val/test)
 - **Cross-validation:** K-fold (usually 5 or 10) for robust performance estimates
-

Traditional Machine Learning

Algorithm Selection Framework

Linear Models (Logistic Regression, Linear Regression)

When to use:

- High-dimensional, sparse data
- Need interpretability
- Limited training data
- Features are linearly separable

Key Parameters:

- **Regularization (C in sklearn):** Higher C = less regularization
 - Start with C=1.0, try [0.01, 0.1, 1, 10, 100]
 - Use L1 for feature selection, L2 for general regularization
- **Solver:** 'liblinear' for small datasets, 'saga' for large datasets

Tree-Based Models (Random Forest, XGBoost)

When to use:

- Mixed data types (numerical + categorical)
- Non-linear relationships
- Don't need strict interpretability
- Robust to outliers

Key Parameters:

- **n_estimators:** Number of trees (start with 100, increase until diminishing returns)
- **max_depth:** Tree depth (3-10 for most cases, deeper = more complex)
- **min_samples_split:** Minimum samples to split node (higher = less overfitting)
- **learning_rate** (XGBoost): Step size (0.01-0.3, lower = better but slower)

SVM (Support Vector Machines)

When to use:

- High-dimensional data
- Clear margin between classes
- Small to medium datasets

Key Parameters:

- **C**: Regularization (1.0 default, higher = less regularization)
- **kernel**: 'rbf' for non-linear, 'linear' for linear separability
- **gamma** (for rbf): Kernel coefficient (higher = more complex decision boundary)

Hyperparameter Tuning Process

Grid Search vs Random Search

- **Grid Search**: Exhaustive, good for few parameters
- **Random Search**: More efficient for many parameters
- **Bayesian Optimization**: Most efficient, use for expensive models

```
python

# Example parameter grids
rf_params = {
    'n_estimators': [100, 200, 500],
    'max_depth': [3, 5, 7, 10, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}
```

Deep Learning

Neural Network Architecture Design

Choosing Network Depth

Why depth matters: Deeper networks can learn more complex patterns

- **Start shallow**: 2-3 layers, increase if underfitting
- **Universal approximation**: Even 1 hidden layer can approximate any function
- **Practical rule**: More data → can handle more depth

Choosing Layer Width

Hidden units per layer:

- **Rule of thumb:** Between input size and output size
- **Start with:** 64-128 neurons for first hidden layer
- **Subsequent layers:** Gradually decrease or keep constant

Activation Functions

Hidden layers:

- **ReLU:** Default choice, computationally efficient
- **Leaky ReLU:** If dying ReLU problem occurs
- **Swish/GELU:** Better performance but more computation

Output layer:

- **Sigmoid:** Binary classification (outputs probability)
- **Softmax:** Multi-class classification (outputs probability distribution)
- **Linear:** Regression (no bounds on output)

Training Parameters Deep Dive

Learning Rate Strategy

Why learning rate is critical: Controls how fast model adapts to problem

- **Too high:** Model oscillates, never converges
- **Too low:** Training too slow, might get stuck in local minima

Learning rate schedules:

- **Constant:** Simple, good starting point (0.001)
- **Step decay:** Reduce by factor every few epochs
- **Cosine annealing:** Smooth reduction following cosine curve
- **Adaptive** (Adam, RMSprop): Automatically adjusts per parameter

Batch Size Effects

Small batches (16-32):

- More frequent updates
- More regularization effect (noise helps escape local minima)
- Less memory usage
- More training time

Large batches (128-512):

- More stable gradients
- Better parallelization
- Risk of overfitting
- Might need higher learning rates

Regularization Techniques

Dropout

How it works: Randomly set neurons to 0 during training

- **Typical values:** 0.2-0.5 for hidden layers
- **Why it works:** Prevents co-adaptation of neurons
- **When to use:** Large networks prone to overfitting

Batch Normalization

Purpose: Normalize inputs to each layer

- **Benefits:** Faster training, allows higher learning rates, regularization effect
- **Placement:** After linear layer, before activation
- **When to use:** Almost always in deep networks

Early Stopping

How it works: Stop training when validation loss stops improving

- **Patience:** Number of epochs to wait (typically 5-20)
 - **Why it works:** Prevents overfitting without reducing model capacity
-

Natural Language Processing

Text Preprocessing Pipeline

Tokenization Strategies

Word-level tokenization:

- **Pros:** Intuitive, preserves meaning
- **Cons:** Large vocabulary, unknown words (OOV) problem
- **When to use:** Small, domain-specific datasets

Subword tokenization (BPE, WordPiece):

- **Pros:** Handles OOV words, smaller vocabulary

- **Cons:** Breaks semantic units
- **When to use:** Large models, multiple languages

Character-level:

- **Pros:** No OOV issues, very small vocabulary
- **Cons:** Longer sequences, loses word-level meaning
- **When to use:** Morphologically rich languages, very small datasets

Vocabulary Management

Vocabulary size considerations:

- **Too small:** Loss of semantic information
- **Too large:** Sparse embeddings, computational cost
- **Sweet spot:** 30K-50K for most applications
- **Special tokens:** <PAD>, <UNK>, <START>, <END>

Sequence Length Handling

Why sequence length matters: Computational complexity, memory usage

- **Padding:** Add special tokens to reach fixed length
- **Truncation:** Cut off sequences that are too long
- **Dynamic batching:** Group similar-length sequences together

Model Architectures

Recurrent Neural Networks (RNN/LSTM/GRU)

When to use:

- Sequential data with temporal dependencies
- Smaller datasets
- Need to process variable-length sequences

Key Parameters:

- **Hidden size:** Dimension of hidden state (128-512 typical)
- **Number of layers:** 1-3 layers usually sufficient
- **Dropout:** Between layers to prevent overfitting
- **Bidirectional:** Process sequence in both directions

LSTM vs GRU:

- **LSTM:** More parameters, better for complex sequences
- **GRU:** Fewer parameters, faster training, similar performance

Transformer Architecture

Why transformers work: Parallel processing, long-range dependencies

- **Self-attention:** Model relationships between all positions
- **Position encoding:** Add positional information to embeddings
- **Multi-head attention:** Multiple attention patterns simultaneously

Key Parameters:

- **Model dimension** (d_{model}): 512, 768 common choices
- **Number of heads:** $d_{\text{model}} / \text{head_dimension}$ (8, 12, 16 typical)
- **Feed-forward dimension:** Usually 4x model dimension
- **Number of layers:** 6-12 for base models, 24+ for large models

Training Considerations

Loss Functions for NLP

Classification tasks:

- **Cross-entropy:** Standard for multi-class classification
- **Focal loss:** For imbalanced datasets
- **Label smoothing:** Reduces overconfidence

Sequence generation:

- **Cross-entropy:** At each position in sequence
- **BLEU/ROUGE:** Evaluation metrics, not differentiable loss

Evaluation Metrics

Classification: Accuracy, F1-score, precision, recall **Generation:** BLEU, ROUGE, METEOR, BERTScore

Language modeling: Perplexity

Large Language Models

Fine-tuning Strategies

Full Fine-tuning

When to use: Sufficient compute resources, domain-specific task

- **Learning rates:** $1e-5$ to $5e-5$ (much smaller than training from scratch)
- **Why low learning rate:** Preserve pre-trained knowledge
- **Batch size:** Often smaller due to memory constraints
- **Epochs:** Typically 2-5 epochs sufficient

Parameter-Efficient Fine-tuning

LoRA (Low-Rank Adaptation)

How it works: Add small trainable matrices to existing weights

- **Rank (r):** Typically 8-64, higher = more capacity
- **Alpha:** Scaling factor, usually 16-32
- **Target modules:** Usually attention weights (q, v, k, o)

Prefix Tuning / P-tuning

How it works: Add trainable prompt tokens to input

- **Prefix length:** 10-100 tokens typically
- **Virtual tokens:** Don't correspond to real vocabulary

Prompt Engineering

Few-shot learning: Provide examples in the prompt

- **Example selection:** Choose diverse, representative examples
- **Order matters:** Performance can vary with example order
- **Template design:** Consistent format helps model performance

Training Specifics for LLMs

Data Preparation

Instruction formatting:

```
### Instruction:  
[Your instruction here]
```

```
### Input:  
[Optional input]
```

```
### Response:  
[Expected response]
```

Chat formatting:

Human: [User message]

Assistant: [Assistant response]

Training Parameters

- **Learning rate scheduling:** Warmup followed by decay
 - **Gradient clipping:** Prevent exploding gradients (clip at 1.0)
 - **Weight decay:** L2 regularization (0.01 typical)
 - **Warmup steps:** 500-2000 steps typically
-

Computer Vision

Data Preprocessing

Image Augmentation

Why augment: Increases dataset size, improves generalization

- **Geometric:** Rotation, scaling, flipping, translation
- **Color:** Brightness, contrast, saturation adjustments
- **Advanced:** Mixup, Cutmix, AutoAugment

Normalization:

- **ImageNet stats:** mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]
- **Why normalize:** Matches pre-training distribution for transfer learning

Handling Different Image Sizes

Resizing strategies:

- **Resize:** Change image dimensions (may distort aspect ratio)
- **Crop:** Maintain aspect ratio, may lose information
- **Pad:** Add borders to reach target size

CNN Architecture Principles

Convolutional Layers

Filter size: 3x3 most common (good efficiency/receptive field tradeoff) **Number of filters:** Typically increases deeper in network (32→64→128→256) **Stride:** Usually 1, use 2 for downsampling instead of pooling

Pooling Layers

Max pooling: Preserves important features, reduces spatial dimensions **Average pooling:** Smoother reduction, less common **Global average pooling:** Replace fully connected layers, reduces parameters

Transfer Learning

Feature extraction: Freeze pre-trained weights, train only classifier **Fine-tuning:** Unfreeze some layers, train with very low learning rate **When to use each:**

- Small dataset, similar domain → Feature extraction
 - Large dataset, different domain → Fine-tuning
 - Large dataset, similar domain → Fine-tuning with higher learning rate
-

Common Parameters Explained

Optimization Parameters

Learning Rate

What it controls: Step size in parameter updates **Typical values:**

- **From scratch:** 0.01-0.1 (SGD), 0.001 (Adam)
- **Fine-tuning:** 1e-5 to 1e-4
- **Finding optimal:** Learning rate finder, start high and decrease

Momentum (SGD)

What it does: Adds "inertia" to parameter updates **Typical value:** 0.9 **Why it helps:** Smooths out oscillations, accelerates convergence

Adam Parameters

- **Beta1** (0.9): Exponential decay rate for first moment estimates
- **Beta2** (0.999): Exponential decay rate for second moment estimates
- **Epsilon** (1e-8): Small constant for numerical stability

Regularization Parameters

L1 vs L2 Regularization

L1 (Lasso):

- Creates sparse weights (feature selection)
- Use when you want interpretability

- Parameter: alpha (higher = more regularization)

L2 (Ridge):

- Prevents large weights
- Keeps all features but shrinks coefficients
- More stable than L1

Dropout Rate

Typical values: 0.2-0.5 **Where to apply:** Usually not on output layer **Adaptive dropout:** Higher rates for larger layers

Model Complexity Parameters

Tree-based Models

- **max_depth:** Deeper = more complex (3-10 typical)
- **min_samples_split:** Higher = less complex (2-20 typical)
- **min_samples_leaf:** Minimum samples in leaf node (1-10 typical)

Neural Networks

- **Hidden units:** More units = more capacity
 - **Number of layers:** More layers = more complex patterns
 - **Weight initialization:** Xavier/He initialization for stable training
-

Debugging & Troubleshooting

Common Training Issues

Model Not Learning (Loss Not Decreasing)

Possible causes and solutions:

1. **Learning rate too low:** Increase learning rate
2. **Learning rate too high:** Decrease learning rate, add learning rate scheduling
3. **Wrong loss function:** Verify loss function matches problem type
4. **Data preprocessing issues:** Check feature scaling, encoding
5. **Vanishing gradients:** Use ReLU activation, batch normalization, residual connections

Overfitting (High Training Accuracy, Low Validation Accuracy)

Solutions:

1. **Add regularization:** Dropout, L1/L2, early stopping
2. **Reduce model complexity:** Fewer parameters, layers, or hidden units
3. **Get more data:** Data augmentation, collect more samples
4. **Cross-validation:** Ensure consistent performance across folds

Underfitting (Low Training and Validation Accuracy)

Solutions:

1. **Increase model complexity:** More layers, hidden units, or parameters
2. **Train longer:** More epochs, lower learning rate
3. **Reduce regularization:** Lower dropout, less L1/L2
4. **Better features:** Feature engineering, domain knowledge
5. **Check data quality:** Ensure labels are correct, features are informative

Unstable Training (Loss Oscillating)

Solutions:

1. **Lower learning rate:** Reduce step size
2. **Gradient clipping:** Prevent exploding gradients
3. **Batch normalization:** Stabilize activations
4. **Different optimizer:** Try Adam instead of SGD, or vice versa

Performance Optimization

Training Speed

1. **Batch size:** Larger batches utilize GPU better
2. **Mixed precision:** Use float16 instead of float32
3. **Data loading:** Use multiple workers, prefetching
4. **Model parallelism:** Split large models across devices

Memory Optimization

1. **Gradient accumulation:** Simulate larger batches
2. **Gradient checkpointing:** Trade compute for memory
3. **Model pruning:** Remove unnecessary parameters
4. **Quantization:** Use lower precision weights

Model Evaluation Best Practices

Metrics Selection

Classification:

- **Balanced datasets:** Accuracy
- **Imbalanced datasets:** F1-score, AUC-ROC, precision/recall
- **Multi-class:** Macro/micro-averaged metrics

Regression:

- **General:** MSE, MAE, R^2
- **Percentage errors:** MAPE (when no zeros in target)
- **Robust to outliers:** MAE, Huber loss

Cross-validation Strategy

- **K-fold:** Standard approach (k=5 or 10)
- **Stratified K-fold:** Maintains class distribution
- **Time series:** Time-based splits, no shuffling
- **Leave-one-out:** For very small datasets

Model Comparison

1. **Statistical significance:** Use paired t-tests or Wilcoxon signed-rank test
2. **Effect size:** Don't just look at p-values
3. **Computational cost:** Training time, inference time, model size
4. **Robustness:** Performance across different data distributions

Final Checklist Before Deployment

1. **Data leakage check:** No future information in features
 2. **Consistent preprocessing:** Same pipeline for training and inference
 3. **Model serialization:** Save model and preprocessing steps
 4. **Performance monitoring:** Set up logging and alerts
 5. **Rollback plan:** Keep previous model version available
 6. **A/B testing framework:** Compare new model against baseline
-

Summary

Machine learning success comes from understanding:

1. **The problem:** Clear definition drives all decisions

2. **The data:** Quality and quantity determine what's possible
3. **The model:** Architecture should match problem complexity
4. **The training:** Parameters control learning dynamics
5. **The evaluation:** Proper validation prevents overfitting to metrics

Remember: Start simple, understand why things work or don't work, then incrementally add complexity.
Every parameter and technique serves a specific purpose in the learning process.